# P2186R2
# Removing Garbage Collection Support

Published Proposal, 2021-04-16

**This version:**
http://wg21.link/P2186R2

**Authors:**
JF Bastien (Woven Planet)
Alisdair Meredith (Bloomberg)

**Audience:**
CWG, LWG

**Project:**
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

**Source:**
github.com/jfbastien/papers/blob/master/source/P2186R2.bs

---

## Table of Contents

## § 1. Abstract

We propose removing (*not* deprecating) C++'s Garbage Collection support. Specifically, these five library functions:

- `declare_reachable`
- `undeclare_reachable`
- `declare_no_pointers`
- `undeclare_no_pointers`
- `get_pointer_safety`

As well as the `pointer_safety` enum, the `__STDCPP_STRICT_POINTER_SAFETY__` macro, and the Core Language wording.

## § 2. Revision History

### § 2.1. r1 ➡ r2

More library names were added to the zombie names section.

### § 2.2. r0 ➡ r1

EWG discussed this paper in a telecon on July 30th 2020, and LEWG discussed this paper in a telecon on December 14th 2020. The following polls were taken:

| | ***SF*** | ***F*** | ***N*** | ***A*** | ***SA*** |
|---|---|---|---|---|---|
| *EWG: Remove (not deprecate) garbage collection support in C++23.* | *3* | *9* | *4* | *0* | *1* |
| *LEWG: Remove (not deprecate) pointer safety in C++23, after moving names to zombie names.* | *10* | *4* | *3* | *0* | *0* |

The library names were added to the zombie names section.

## § 3. History

Minimal support for Garbage Collection was added to C++0x in 2008 by [N2670]. The main addition was the concept of "strict pointer safety", as well as library support for this pointer safety. Most of the rationale leading to the wording in this paper is captured in the two preceding proposals that merged to form this minimal paper, [N2310] and [N2585].

There have been successful garbage collectors for C++, for example the Boehm GC as well as Garbage Collectors in language virtual machines where the VM is implemented in C++, to support a garbage-collected language. This allows the implementation language to reference objects in the garbage collected language, and lets them interface very closely. You're likely reading this paper in such a virtual machine, implemented in C++, with support for garbage collection: JavaScript VMs do this. Similarly, you've probably played games which mix C++ and C# using the Unity game engine, which relies on the Boehm GC.

Example of virtual machines written in C++ with support for garbage collection include:

- WebKit's JavaScriptCore use a garbage collector called Riptide.
- Chromium's Blink GC called Oilpan. The V8 blog has a good overview of Oilpan. The V8 JavaScript engine used by Chromium also has its own garbage collector called Orinoco.
- Firefox's SpiderMonkey JavaScript engine also has a garbage collector.
- Lua and LuaJIT use garbage collection.

As you can see from their documentation, each garbage collector has its own set of design criteria which influence how the language itself is implemented, and how the C++ runtime is written to obey

the chosen design. These languages use similar ideas, but the design is different in each case, and the constraints on C++ code are different.

## § 4. Issues with the Current Specification

We illustrate a few problems with the current specification as it stands, in some cases where the current specification is overly restrictive, and in others where it falls short.

### § 4.1. Safely Derived Pointers

The complete list of ways to create a safely-derived pointer is itemized in **[basic.stc.dynamic.safety]** ¶2. The list is mostly manipulation of existing safely-derived pointers, where the only way to create the initial safely-derived pointer is through a call to one of two specified overloads of global `::operator new`. It does not have an escape hatch for implementation-defined behavior adding additional ways to create a safely derived pointer. In particular, calls to global array-`new`, or no-throw `new`, do not produce safely-derived pointers unless defined to call one of the two specified overloads.

However, the most troubling example is using in-place new to create object in local arrays, a common strategy to avoid unnecessary heap usage:

```
#include <new>

int main() {
   char buffer[sizeof(int)] alignas(int); // automatic storage duration
   void *ptr = buffer;
   int *pint = new(ptr) int(0);           // dynamic storage duration
   return *pint;                          // UB with strict pointer safe
}
```

Instinctively, we might reach for `std::declare_reachable` to solve such matters ourselves, at the expense of complicating portable code for the befit of well-defined behavior on strict pointer safety systems. Alas! This does not work, as the precondition on `std::declare_reachable` is that the supplied pointer be safely-derived—the very problem we are trying to solve by using this function! Even if that precondition were relaxed, there would be a problem calling `std::undeclare_reachable` before the function returns.

## § 4.2. Allocators

Safely derived pointers to dynamic memory cannot be provided other than by calls to `::operator new(std::size_t)` or `::operator new(std::size_t, std::align_val_t)`, see **[basic.stc.dynamic.safety]** ¶1. This means we have no support for OS memory allocation functions, such as `VirtualAlloc` or `HeapAlloc` on Windows, or use of memory mapped files for interprocess communication.

Custom memory allocation, as might be supplied by a type that meets the allocator requirements, or implements the `prm::memory_resource` interface, typically rely on such allocation subsystems, and would need some as yet unspecified scheme to indicate that they hold valid memory that could hold pointers to live objects. Note that simply calling `declare_reachable` on every attempt to construct an object through such an allocator is not sufficient, as that function has a precondition that the pointer argument is safely-derived—exactly the problem we are trying to solve.

## § 4.3. Replacement `operator new`

The only two library functions guaranteed to return a safely-derived pointer are *replaceable*, but there is no mention in the library specification of what it means to replace these functions on an implementation with strict pointer safety, or whether the replacement might in turn might introduce strict pointer safety into the program.

Further, according to **[expr.new]** ¶12, "An implementation is allowed to omit a call to a replaceable global allocation function. When it does so, the storage is instead provided by the implementation or provided by extending the allocation of another new-expression." If this is intended that these extended allocations be constrained to return a safely pointer on implementations with strict pointer safety, a note (if not normative text) would be helpful.

## § 4.4. `constexpr` Allocation

Does compile-time (`constexpr`) allocation by the language have strict, relaxed, or preferred memory safety? In practice, the current answer is largely irrelevant, as the only supported compile-time allocators call the global `::operator new` function, which by definition returns safely-derived pointers. Similarly, the masking and unmasking operations that might produce non-safely-derived pointers are not supported during constant evaluation. However, do note that the library facilities for handling pointer safety are not marked as `constexpr`, so any library containers that make an effort to tune for performance on a garbage collected implementation must also guard such calls with a check for if (`std::is_constant_evaluated()`), genuinely avoid pointer masking tricks, and prepare for

[P1974R0] Non-transient `constexpr` allocation using `propconst` providing support for `constinit` objects allocated at compile-time, but used and extended at runtime.

## § 4.5. Core versus Library Wording

The core language talks about traceable pointer *objects* while the library uses the term traceable pointer *location*. This latter term is never defined, although the inference from cross-references is that they may be intended to mean the same thing. We should use the core term throughout the library as well, or more clearly define their relationship if distinct terms are intended.

Our current best guess is that the two terms are intended to be distinct. From the usage in **[util.dynamic.safety]** ¶11, it seems that a traceable pointer location is a possible property of the value stored in a traceable pointer object, such that all traceable pointer objects assume the traceable pointer location property unless `declare_no_pointers` is called.

## § 4.6. `signed char` and `std::byte`

According to **[basic.stc.dynamic.safety]** ¶3, a *traceable pointer object* may be "a sequence of elements in an array of narrow character type, where the size and alignment of the sequence match those of some object pointer type." This seems reasonable for types `char` and `unsigned char`, which have special dispensation to be trafficked as raw memory. However, it may be more surprising to find this applies to arrays of `signed char` and `char8_t` as well, which other than in this one paragraph, have no such memory aliasing properties. Similarly, it is surprising that arrays of `std::byte`, a type deliberately introduced to describe raw memory, do not have this property.

## § 4.7. Preferred Pointer Safety

A call to `std::get_pointer_safety` can return a value indicating `strict`, `relaxed`, or `preferred`. It is not clear what the difference between `preferred` and `relaxed` memory safety is. From a core wording perspective, there is no difference, so the domain of well-defined behavior does not change. Other than this one mention on the specification for the `get_pointer_safety` there is no description of what it means, and how programs should behave differently when informed of this. It appears to raise confusion, for no clear benefit. For example, should a program with concerns about `strict` pointer safety check that an implementation has `relaxed` pointer safety, or merely that it does not have `strict` pointer safety? While these two questions are equivalent according to the core language specification, they can produce different results when querying the library API intended for this purpose.

## § 4.8. C Compatibility

Despite a decade of standards since C++11 (C11, C18, and the pending C2X), there has been no enthusiasm in WG14 to add similar garbage collection support to the C language.

## § 4.9. `malloc` and Related Functions

In order to achieve binary compatibility with C code, an implementation must assume that all memory returned from a call to a C allocation function is implicitly declared reachable. It is not clear how this differs from being a safely-derived pointer, as a pointer to an object that has been declared reachable is never treated as invalid due to not being safely-derived. We suspect the intent is that such memory is to be treated similarly to that for automatic, static, and thread-local storage duration objects, other than the obscure normative text that says such pointers can be passed to `declare_unreachable` which must somehow contrive to support this, and most likely ignore that pointer in such cases. This seems an obscurely specific way to permit a subset of pointers to be validly passed to a function that has no business seeing them. It would be much simpler to make the precondition on `declare_unreachable` that there be a matching call to `declare_reachable`, or to use another term to describe the reachability that comes from a call to the C allocation APIs.

# § 5. Rationale

Based on the above history, Garbage Collection in C++ is clearly useful for particular applications.

However, Garbage Collection as specified by the Standard is not useful for those applications. In fact, the authors are not aware of any implementations of the strict pointer safety facility. Unsurprisingly, the authors are not aware of any uses either. Indeed, ISOCpp code search only finds hits in GCC and LLVM. Similarly, CppReference tells us that implementations all offer no support for this feature. Finally, the specification falls short in many ways as outlined above.

It’s unclear whether the Standard should make Garbage Collection an (optional?) first-class feature, because the aforementioned language VMs function differently from each other. What is clear is that the current specification isn’t helpful for any of them. The library facilities are clearly unused. The Core wording intends to offer minimal guarantees for garbage collectors, but doesn’t actually provide any actionable guidance to implementations, even if "strict" pointer safety were offered. Even then, libc++, libstdc++, and Microsoft’s Standard Library all offer relaxed pointer safety and not strict pointer safety. In other words, the Core wording currently provides no restrictions on implementations, and the implementations nonetheless decided to go for the weaker "relaxed" option. Further, garbage collectors rely on other quality-of-implementations factors which Core wording is silent on.

Finally, existing Standard Library implementations would need to be significantly changed were they to attempt supporting strict pointer safety, for example by marking regions memory pointer-free with `declare_no_pointers` in containers such as `std::vector<int>`. Asking a Standard Library implementation to provide good support for strict pointer safety is tantamount to doubling the number of dialects that should be supported (including existing unofficial dialects such as without exceptions and type information).

This status-quo hasn't changed in 12 years. The maintenance burden on the Standard is near minimal, and we hope the Committee spends almost no time agreeing to remove this unused and unimplemented feature, despite its origins being well-intended and the target use-case still being relevant. Indeed, the current specification simply missed the mark, and will not be missed.

We therefore propose outright removal instead of deprecation because lack of implementation and usage makes deprecation moot.

## § 6. Other Concerns

There are several other features in C++ that deal with the validity of pointers, or allocating memory. After some consideration, the following features were reviewed, but determined to have no impact on the strict pointer safety model. They are listed here so that the reader is aware that they were not overlooked.

- `std::launder`
- allocation for coroutines
- allocation for exception objects

## § 7. Proposal

Remove all of **[basic.stc.dynamic.safety]** as follows:

A *traceable pointer object* is

- an object of an object pointer type, or
- an object of an integral type that is at least as large as `std::intptr_t`, or
- a sequence of elements in an array of narrow character type, where the size and alignment of the sequence match those of some object pointer type.

A pointer value is a *safely-derived pointer* to an object with dynamic storage duration only if the pointer value has an object pointer type and is one of the following:

- the value returned by a call to the C++ standard library implementation of `::operator new(std::size_t)` or `::operator new(std::size_t, std::align_val_t)`
- the result of taking the address of an object (or one of its subobjects) designated by an lvalue resulting from indirection through a safely-derived pointer value;
- the result of well-defined pointer arithmetic using a safely-derived pointer value;
- the result of a well-defined pointer conversion of a safely-derived pointer value;
- the result of a `reinterpret_cast` of a safely-derived pointer value;
- the result of a `reinterpret_cast` of an integer representation of a safely-derived pointer value;
- the value of an object whose value was copied from a traceable pointer object, where at the time of the copy the source object contained a copy of a safely-derived pointer value.

An integer value is an *integer representation of a safely-derived pointer* only if its type is at least as large as `std::intptr_t` and it is one of the following:

- the result of a `reinterpret_cast` of a safely-derived pointer value;
- the result of a valid conversion of an integer representation of a safely-derived pointer value;
- the value of an object whose value was copied from a traceable pointer object, where at the time of the copy the source object contained an integer representation of a safely-derived pointer value;
- the result of an additive or bitwise operation, one of whose operands is an integer representation of a safely-derived pointer value `P`, if that result converted by `reinterpret_cast<void*>` would compare equal to a safely-derived pointer computable from `reinterpret_cast<void*>(P)`.

An implementation may have `relaxed pointer safety`, in which case the validity of a pointer value does not depend on whether it is a safely-derived pointer value. Alternatively, an

> ~~implementation may have `strict pointer safety`, in which case a pointer value referring to an object with dynamic storage duration that is not a safely-derived pointer value is an invalid pointer value unless the referenced complete object has previously been declared reachable. [*Note*: The effect of using an invalid pointer value (including passing it to a deallocation function) is undefined. This is true even if the unsafely-derived pointer value might compare equal to some safely-derived pointer value. — *end note* ] It is implementation defined whether an implementation has relaxed or strict pointer safety.~~

In **[expr.reinterpret.cast]**, remove the note as follows:

> A value of integral type or enumeration type can be explicitly converted to a pointer. A pointer converted to an integer of sufficient size (if any such exists on the implementation) and back to the same pointer type will have its original value; mappings between pointers and integers are otherwise implementation-defined. ~~[*Note:* Except as described in **[basic.stc.dynamic.safety]**, the result of such a conversion will not be a safely-derived pointer value. — *end note* ]~~

In **[new.delete]**, remove the six instances of the pointer safety precondition on `operator delete` overloads as follows:

```
void operator delete(void* ptr) noexcept;
void operator delete(void* ptr, std::size_t size) noexcept;
void operator delete(void* ptr, std::align_val_t alignment) noexcept;
void operator delete(void* ptr, std::size_t size, std::align_val_t align

void operator delete(void* ptr, const std::nothrow_t&) noexcept;
void operator delete(void* ptr, std::align_val_t alignment, const std::n

void operator delete[](void* ptr) noexcept;
void operator delete[](void* ptr, std::size_t size) noexcept;
void operator delete[](void* ptr, std::align_val_t alignment) noexcept;
void operator delete[](void* ptr, std::size_t size, std::align_val_t ali

void operator delete[](void* ptr, const std::nothrow_t&) noexcept;
void operator delete[](void* ptr, std::align_val_t alignment, const std:

void operator delete(void* ptr, void*) noexcept;

void operator delete[](void* ptr, void*) noexcept;
```

~~*Preconditions:* If an implementation has strict pointer safety **[basic.stc.dynamic.safety]** then ptr is a safely-derived pointer.~~

In **[memory.syn]**, remove as follows:

```
// 20.10.5, pointer safety
enum class pointer_safety { relaxed, preferred, strict };
void declare_reachable(void* p);
template<class T>
T* undeclare_reachable(T* p);
void declare_no_pointers(char* p, size_t n);
void undeclare_no_pointers(char* p, size_t n);
pointer_safety get_pointer_safety() noexcept;
```

Remove all of **[util.dynamic.safety]**, and associated implementation-defined behavior in the annex.

In **[cpp.predefined]**, remove as follows:

~~`__STDCPP_STRICT_POINTER_SAFETY__`~~
~~Defined, and has the value integer literal `1`, if and only if the implementation has strict pointer safety.~~

In **[c.malloc]**, remove as follows:

~~Storage allocated directly with these functions is implicitly declared reachable on allocation, ceases to be declared reachable on deallocation, and need not cease to be declared reachable as the result of an `undeclare_reachable()` call. [ *Note:* This allows existing C libraries to remain unaffected by restrictions on pointers that are not safely derived, at the expense of providing far fewer garbage collection and leak detection options for `malloc()`-allocated objects. It also allows `malloc()` to be implemented with a separate allocation arena, bypassing the normal `declare_reachable()` implementation. The above functions should never intentionally be used as a replacement for `declare_reachable()`, and newly written code is strongly encouraged to treat memory allocated with these functions as though it were allocated with `operator new`. — *end note* ]~~

In **[zombie.names]**, edit the first paragraph as follows:

In namespace `std`, the following names are reserved for previous standardization:

- `declare_reachable`
- `undeclare_reachable`
- `declare_no_pointers`
- `undeclare_no_pointers`
- `get_pointer_safety`
- `pointer_safety`

In **[zombie.names]**, edit the second paragraph as follows:

The following names are reserved as ~~member types~~ members for previous standardization, and may not be used as a name for object-like macros in portable code:

- `preferred`
- `strict`

Do not add `relaxed` to this list, it is already a reserved member of `enum class memory_order` as of C++20.

# § References

## § Informative References


**[N2310]**

H.-J. Boehm, M. Spertus. Transparent Programmer-Directed Garbage Collection for C++. 20 June 2007. URL: https://wg21.link/n2310

**[N2585]**

H. Boehm, M. Spertus. Minimal Support for Garbage Collection and Reachability-Based Leak Detection. 16 March 2008. URL: https://wg21.link/n2585

**[N2670]**

H.-J. Boehm, M. Spertus, C. Nelson. Minimal Support for Garbage Collection and Reachability-Based Leak Detection (revised). 13 June 2008. URL: https://wg21.link/n2670

**[P1974R0]**

Jeff Snyder, Louis Dionne, Daveed Vandevoorde. Non-transient constexpr allocation using propconst. 15 May 2020. URL: https://wg21.link/p1974r0